

Diabetes Risk Prediction (N/P/Y): Classification and Clustering Analysis

1. Introduction

This report investigates a multi-class diabetes prediction task using a clinical dataset containing demographic information and laboratory measurements. The objective is to predict patient diabetes status across three categories Non-diabetic (N), Prediabetic (P), and Diabetic (Y) and to compare supervised classification approaches with unsupervised clustering. The study follows a standard machine learning workflow including data cleaning, exploratory data analysis (EDA), preprocessing, model training, evaluation, and error analysis to identify which methods perform best and why.

2. Dataset Description

The dataset consists of 1000 patient records with demographic variables (Age, Gender), laboratory biomarkers (e.g., HbA1c, lipid profile, Urea, Creatinine), and BMI. The target variable CLASS contains three labels (N, P, Y). Identifier-like columns (e.g., ID, No_Pation) were excluded from modeling to prevent non-informative patterns.

	ID	No_Pation	Gender	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLDL	BMI	CLASS
count	1000.000000	1.000000e+03	1000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000
unique	NaN	NaN	2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	3
top	NaN	NaN	M	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Y
freq	NaN	NaN	565	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	844
mean	340.500000	2.705514e+05	NaN	53.528000	5.124743	68.943000	8.281160	4.862820	2.349610	1.204750	2.609790	1.854700	29.578020	NaN
std	240.397673	3.380758e+06	NaN	8.799241	2.935165	59.984747	2.534003	1.301738	1.401176	0.660414	1.115102	3.663599	4.962388	NaN
min	1.000000	1.230000e+02	NaN	20.000000	0.500000	6.000000	0.900000	0.000000	0.300000	0.200000	0.300000	0.100000	19.000000	NaN
25%	125.750000	2.406375e+04	NaN	51.000000	3.700000	48.000000	6.500000	4.000000	1.500000	0.900000	1.800000	0.700000	26.000000	NaN
50%	300.500000	3.439550e+04	NaN	55.000000	4.600000	60.000000	8.000000	4.800000	2.000000	1.100000	2.500000	0.900000	30.000000	NaN
75%	550.250000	4.538425e+04	NaN	59.000000	5.700000	73.000000	10.200000	5.600000	2.900000	1.300000	3.300000	1.500000	33.000000	NaN
max	800.000000	7.543566e+07	NaN	79.000000	38.900000	800.000000	16.000000	10.300000	13.800000	9.900000	9.900000	35.000000	47.750000	NaN

3. Data Quality & Cleaning

3.1 Dataset Overview

```

<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   ID           1000 non-null   int64  
1   No_Pation    1000 non-null   int64  
2   Gender       1000 non-null   str     
3   AGE          1000 non-null   int64  
4   Urea         1000 non-null   float64 
5   Cr           1000 non-null   int64  
6   HbA1c        1000 non-null   float64 
7   Chol         1000 non-null   float64 
8   TG           1000 non-null   float64 
9   HDL          1000 non-null   float64 
10  LDL          1000 non-null   float64 
11  VLDL         1000 non-null   float64 
12  BMI          1000 non-null   float64 
13  CLASS        1000 non-null   str     
dtypes: float64(8), int64(4), str(2)
memory usage: 109.5 KB

```

- Total rows: 1000
- Total columns: 14
- Memory usage: ~109.5 KB (very small; easy to process)

This indicates that the dataset is lightweight and suitable for exploratory analysis and machine learning experiments.

Column Breakdown

1. Numerical Columns

There are 12 numerical columns, divided into:

Integer columns (4):

- ID
- No_Pation
- AGE
- Cr

These contain whole-number values, although ID and No_Pation are identifiers and will be removed later because they do not contribute predictive value.

Float columns (8):

- Urea

- HbA1c
- Chol
- TG
- HDL
- LDL
- VLDL
- BMI

These represent continuous biochemical measurements and body metrics.

2. Categorical (Object) Columns (2):

- Gender
- CLASS

Gender is a demographic variable, while CLASS is the target label for prediction.

Because both are stored as strings (object dtype), they will require encoding during preprocessing.

Missing Values

All columns show 1000 non-null entries, meaning:

- There are no missing values in the dataset.
- No imputation is needed at this stage.
- This simplifies preprocessing significantly.

Interpretation & Significance

1. Consistent Data Types:

Numeric columns are correctly typed as int64 or float64, which makes them compatible with ML algorithms.

2. Categorical Columns Identified:

Gender and CLASS will need encoding (OneHotEncoding, LabelEncoding) before model training.

3. No Missing Values:

Prevents complications such as imputation strategies, allowing the model workflow to remain straightforward.

4. ID Columns Detected:

ID and No_Pation serve as identifiers. They must be dropped because:

- They do not add predictive value

- They can introduce noise
- They may cause data leakage

5. **Memory Efficiency:**

At only 109.5 KB, the dataset is extremely efficient for operations and ideal for training and testing ML models rapidly.

3.2 Descriptive Statistics Summary

The `df.describe()` output highlights the central tendency and spread of numeric features:

- AGE: Median 55, range 20–79 → middle-aged population typical for diabetes studies
- HbA1c: Median 8.0, range 0.9–16 → broad glycemic spectrum across N, P, Y classes
- BMI: Median 30, range 19–47.75 → majority overweight/obese
- Urea & Creatinine: Heavy right-skew with extreme maxima (38.9, 800) → kidney stress indicators
- TG & VLDL: Strongly skewed with large outliers → lipid abnormalities
- Chol, LDL, HDL: Moderate variability, realistic clinical ranges
- ID & No_Patien: Identifier columns; removed during preprocessing

These statistics confirm skewness, outliers, and clinically meaningful trends that guide preprocessing and modeling.

3.3 Clean Column Text

This step standardizes column names and categorical text fields to ensure consistency prior to preprocessing:

- Column names are trimmed to remove whitespace and prevent referencing errors.
- CLASS values are converted to clean strings and stripped of spaces to avoid duplicated categories (e.g., "Y" vs "Y ").
- Gender values are standardized by removing whitespace and converting to uppercase (M/F), ensuring consistent encoding.

These cleaning operations prevent categorical duplication, reduce preprocessing errors, and improve the reliability of downstream machine learning tasks.

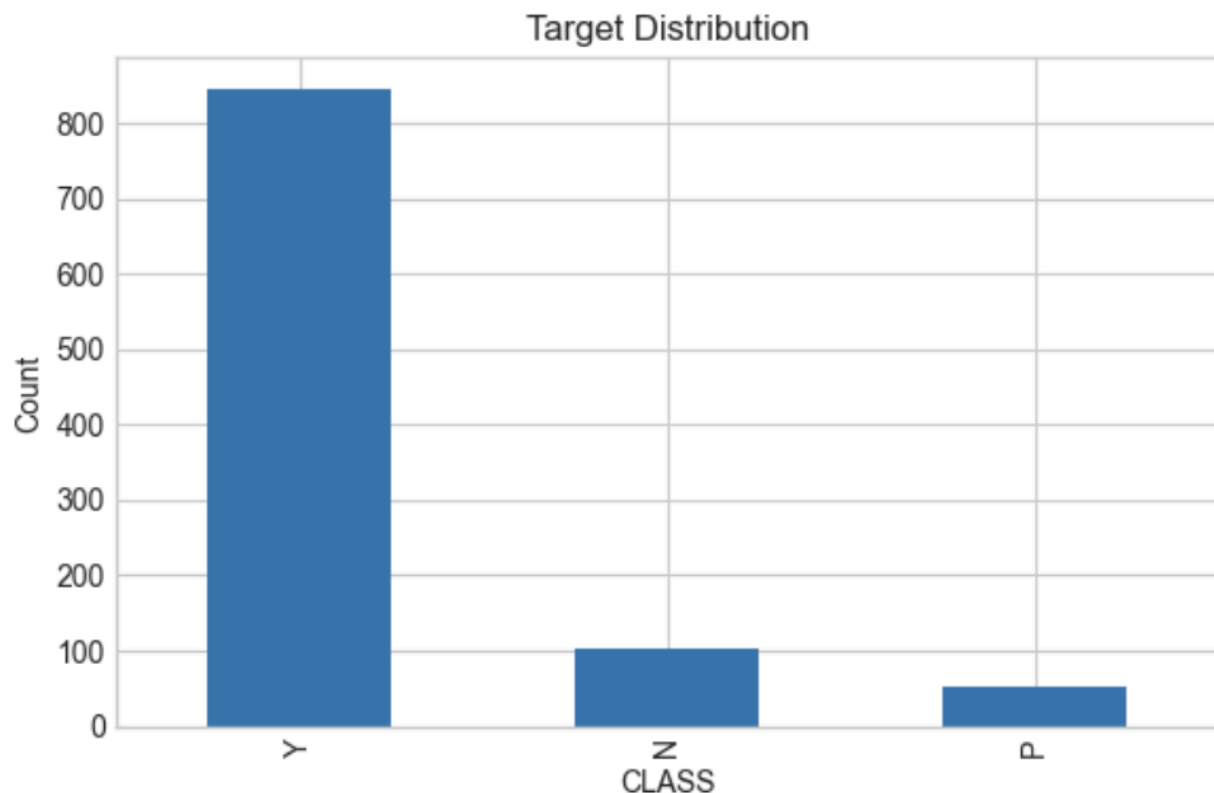
3.4 Feature & Target Split

- Dropped ID columns: ID, No_Patien.

- Feature set contains 11 columns: Gender, AGE, Urea, Cr, HbA1c, Chol, TG, HDL, LDL, VLDL, BMI.
- Target column (CLASS) is kept separately for analysis and modeling.

4. EDA Findings

4.1 Target Distribution

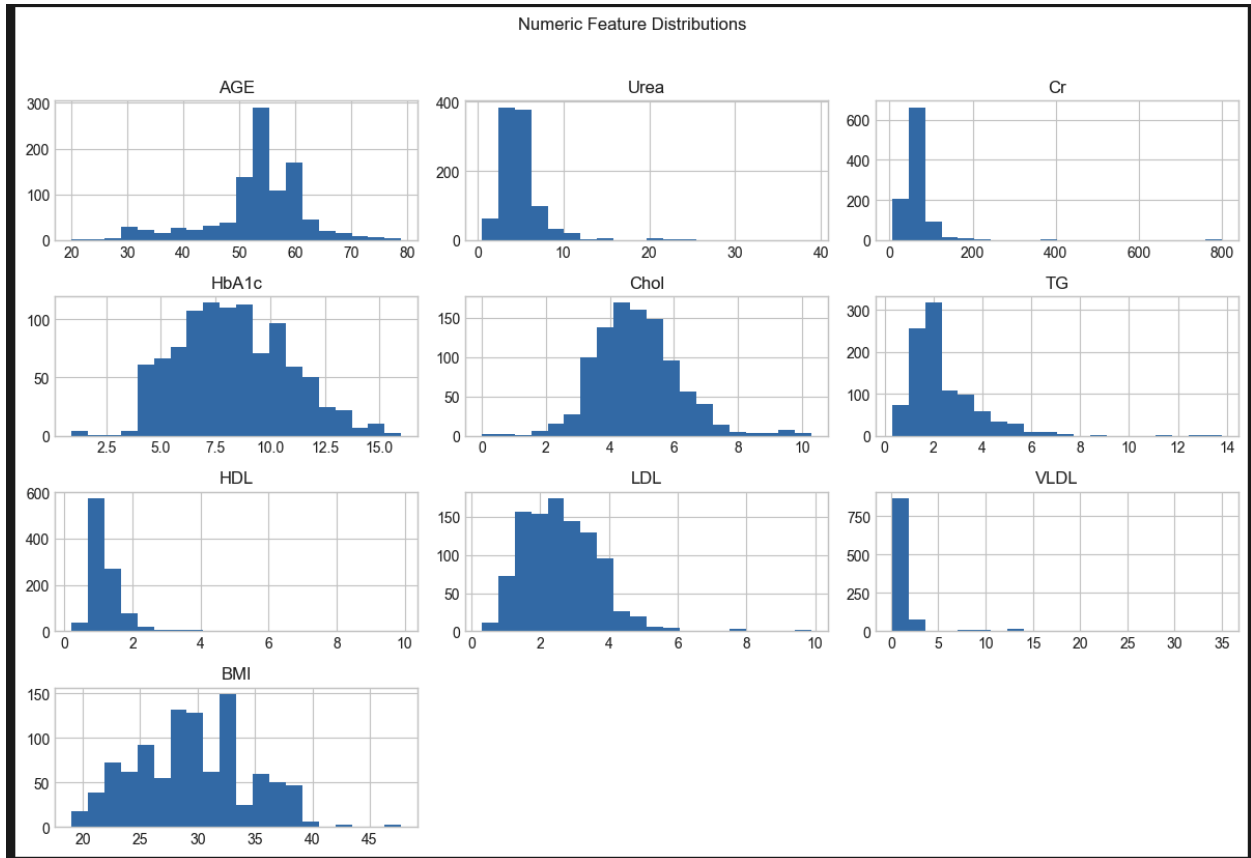


The dataset is highly imbalanced, with the vast majority of samples belonging to the diabetic (Y) class. Specifically:

- Y (Diabetic): 844 samples
- N (Non-diabetic): 103 samples
- P (Prediabetic): 53 samples

This imbalance indicates that accuracy alone is not a reliable metric for evaluating models. Therefore, macro-F1 and weighted-F1 scores are required to assess performance fairly, especially for the minority class P, which is the most difficult to classify.

4.2 Univariate Distribution of Numeric Features



A series of histograms were analyzed to understand the distribution, shape, and outlier behavior of each numerical variable:

AGE

- Values range mostly from 35 to 65.
- Follows a roughly normal distribution.
- Reflects a middle-aged population where diabetes incidence is typically higher.

Urea

- Strong right skew with a typical range of 3–8, but several outliers reach above 30.
- Suggests potential kidney function irregularities in some individuals.

Creatinine (Cr)

- Extremely heavy-tailed distribution with outliers reaching as high as ~800.

- Most values remain under 100.
- Very high levels are consistent with renal complications often associated with diabetes.

HbA1c

- Displays a wide distribution from 4 to 16.
- Shows clear representation of all diabetes stages.
- Serves as the primary biochemical indicator of diabetic state.

Cholesterol (Chol) and LDL

- Both follow moderately symmetric distributions around 4–5 mmol/L.
- Show reasonable variability with occasional high values.

Triglycerides (TG) and VLDL

- Highly right-skewed with numerous outliers.
- Indicative of lipid abnormalities frequently observed in diabetic patients.

HDL

- Concentrated around 1 mmol/L with moderate variability.
- Extreme lows and highs are observed across classes.

BMI

- Centered around 26–32, with the diabetic group tending toward higher BMI values.
- A small number of extremely high BMI outliers are present.

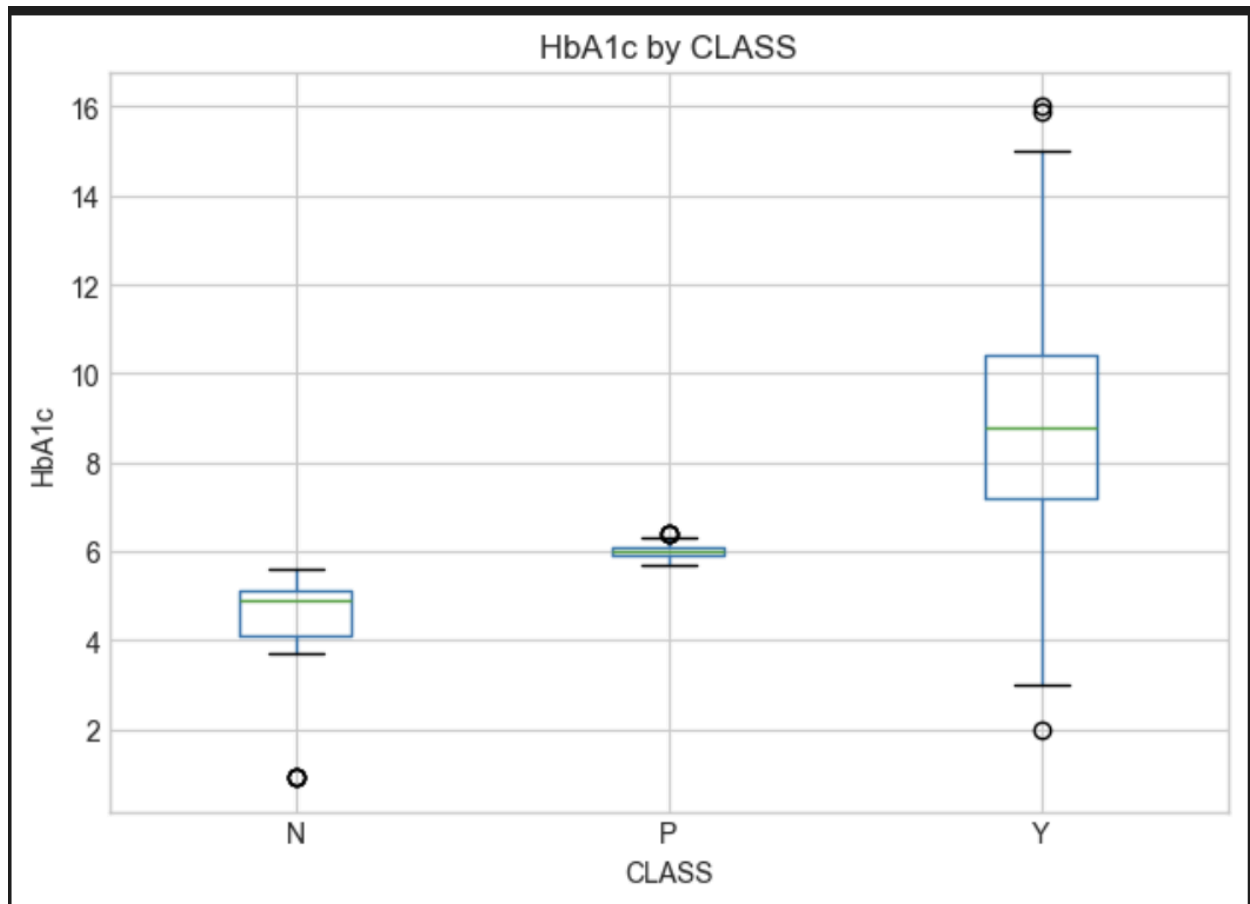
Key Insight:

Several features display significant skewness and outlier sensitivity, making feature scaling and possibly transformation essential, especially for distance-based models (KNN, SVM, KMeans).

4.3 Boxplots: Feature Distribution Across CLASS

Boxplots were used to analyze how each numeric feature differs across the three target classes (N, P, Y).

HbA1c by CLASS

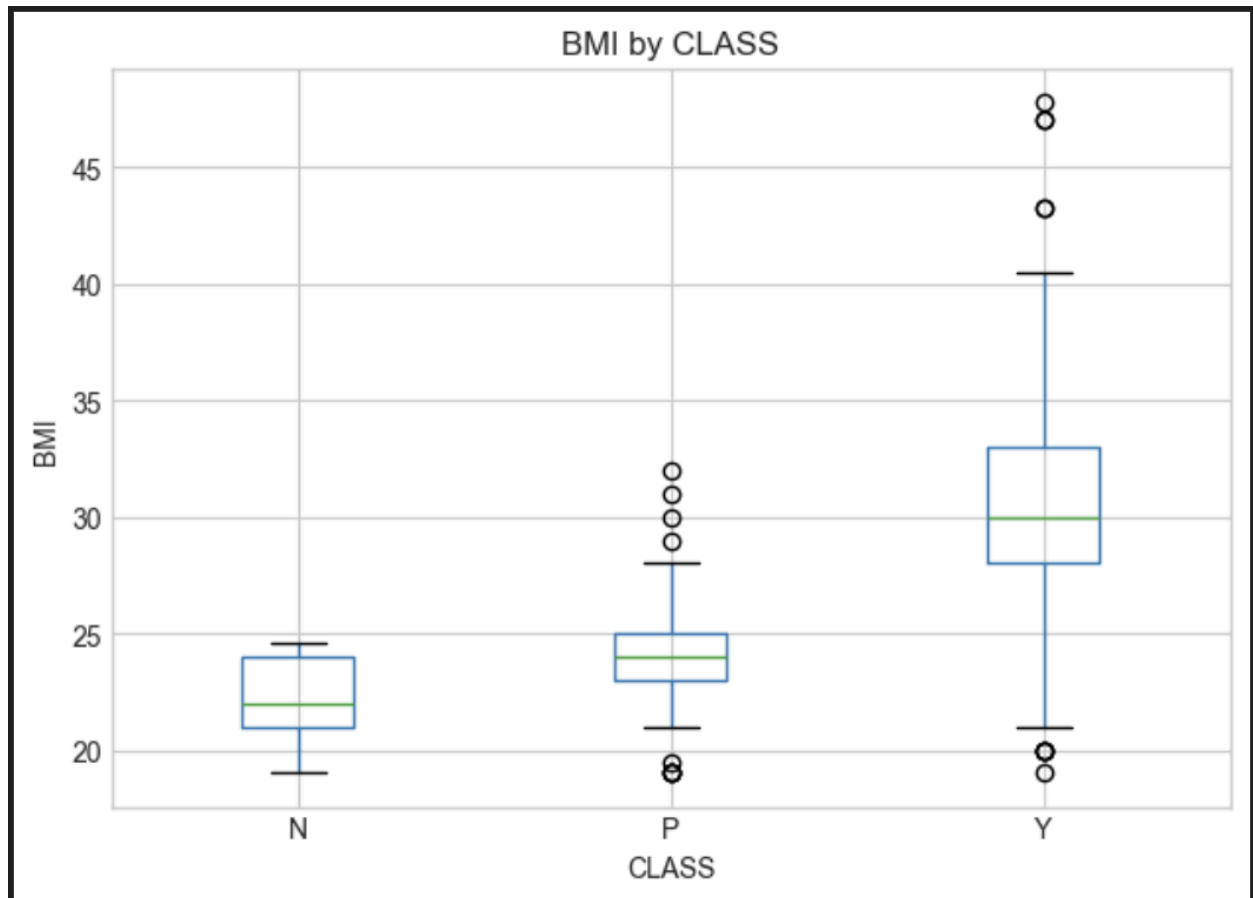


HbA1c demonstrates excellent class separation:

- N: Median around 5
- P: Median around 6
- Y: Median around 9–10 with values stretching up to 16

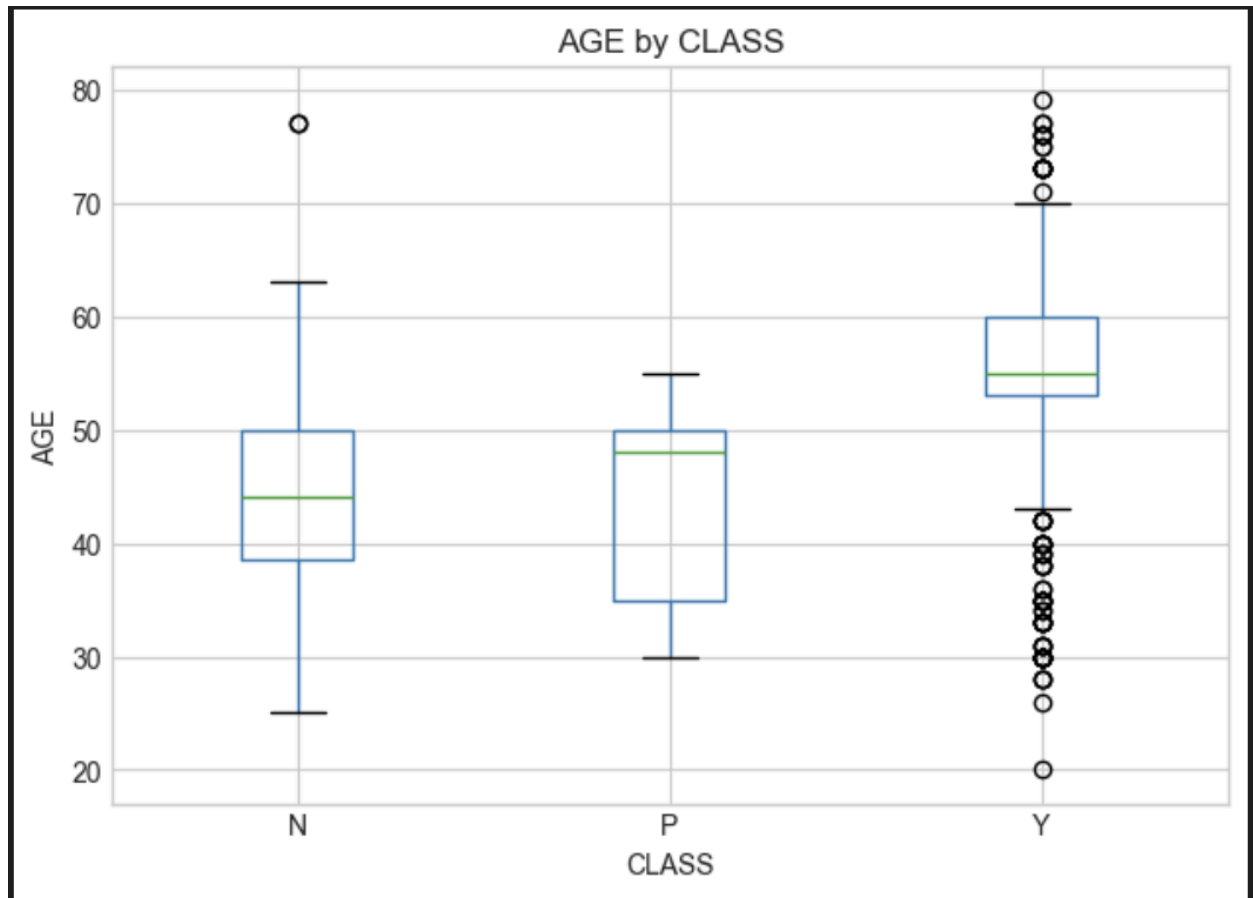
There is minimal overlap among the three groups, making HbA1c the most predictive feature of diabetic status. This explains why models achieve high accuracy.

BMI by CLASS



- BMI increases progressively from N -> P -> Y.
- Diabetic patients have noticeably higher BMI compared to non-diabetic individuals.
- Supports the well-known association between obesity and diabetes risk.

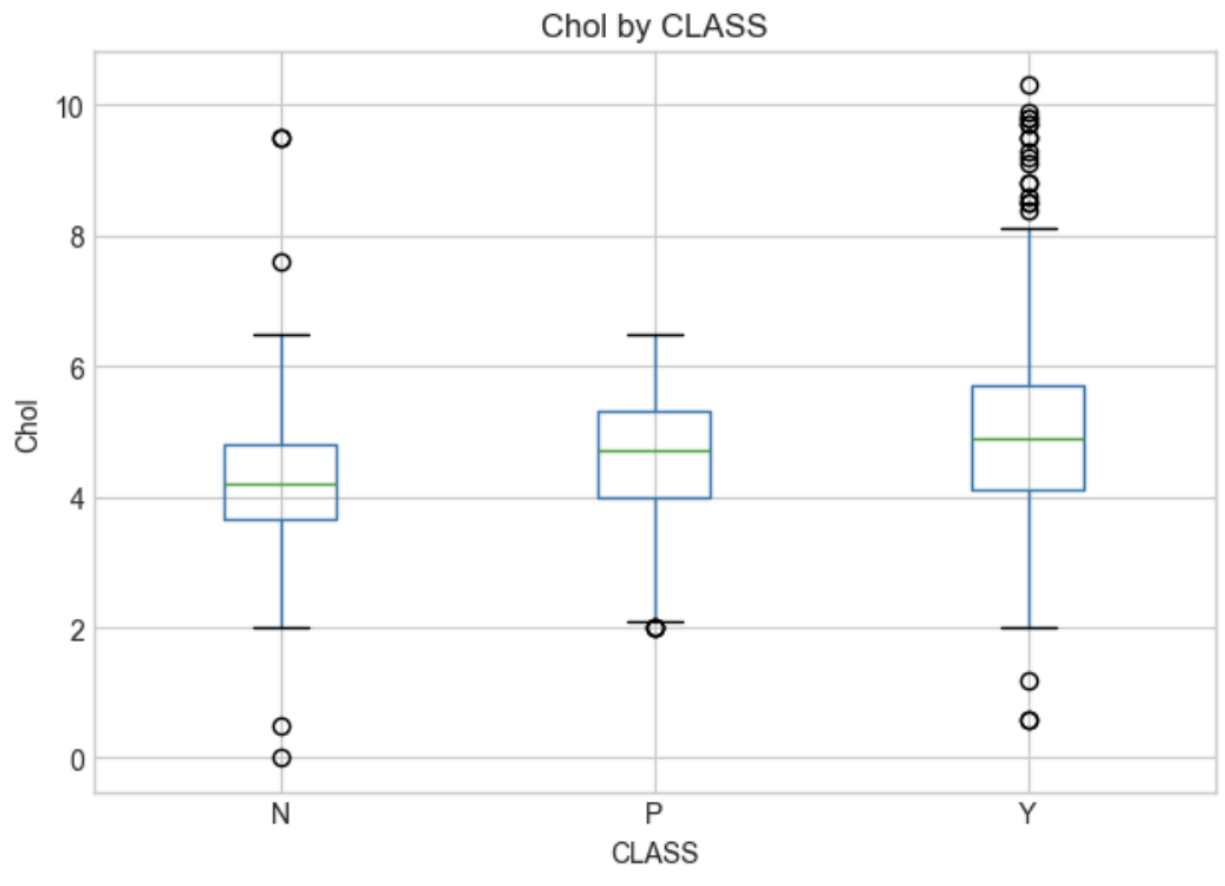
AGE by CLASS

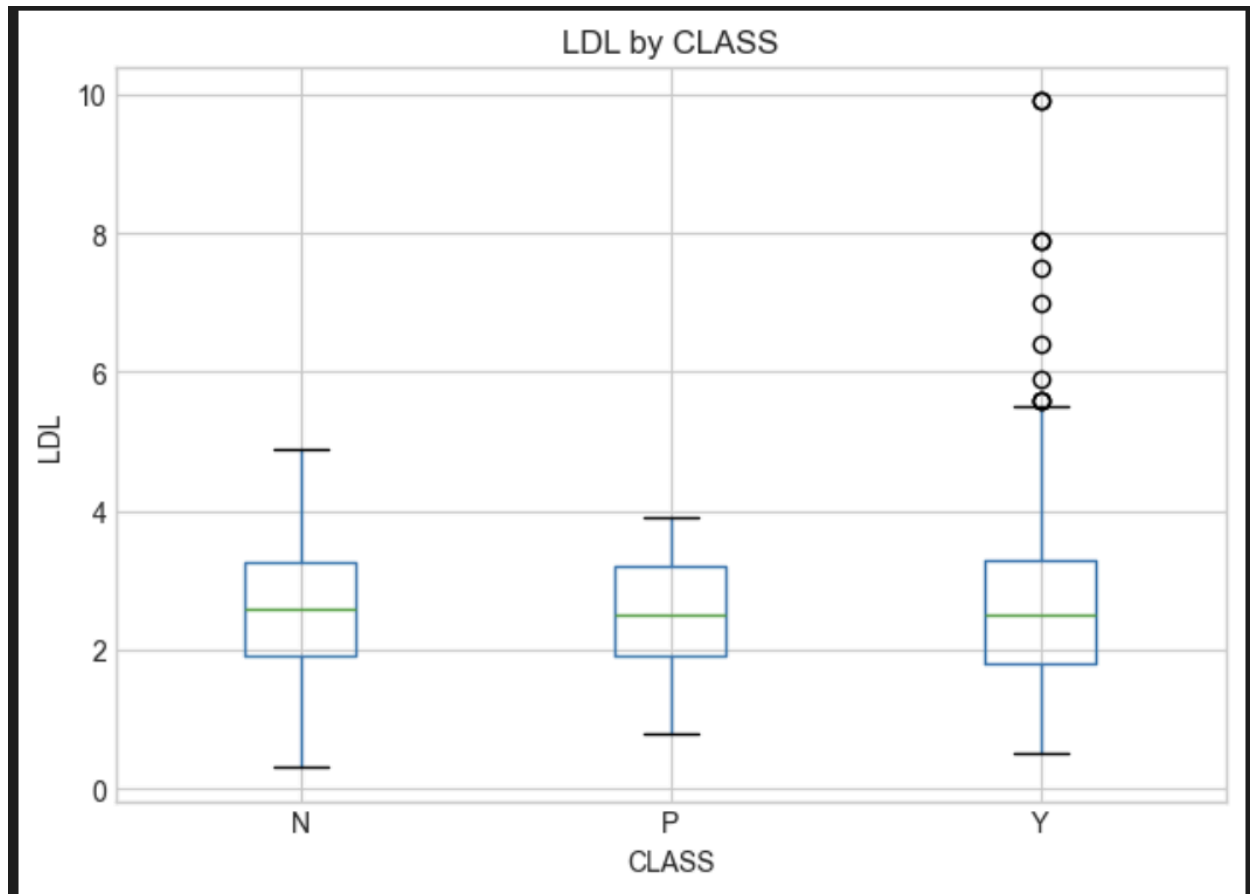


- N group is the youngest on average.
- Y group is the oldest.
- P sits between them.

Age is a meaningful predictor but not strongly discriminative by itself.

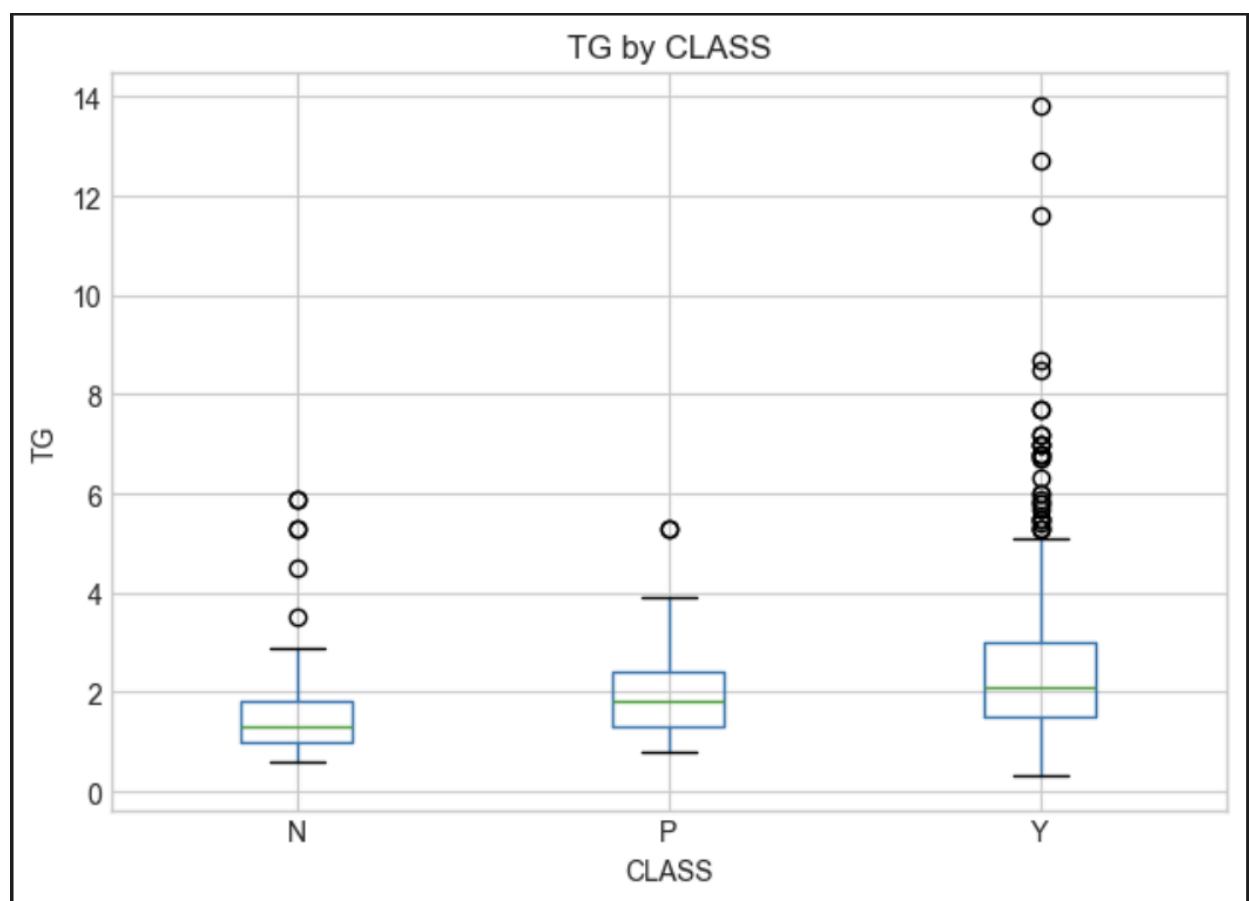
Cholesterol (Chol) and LDL by CLASS

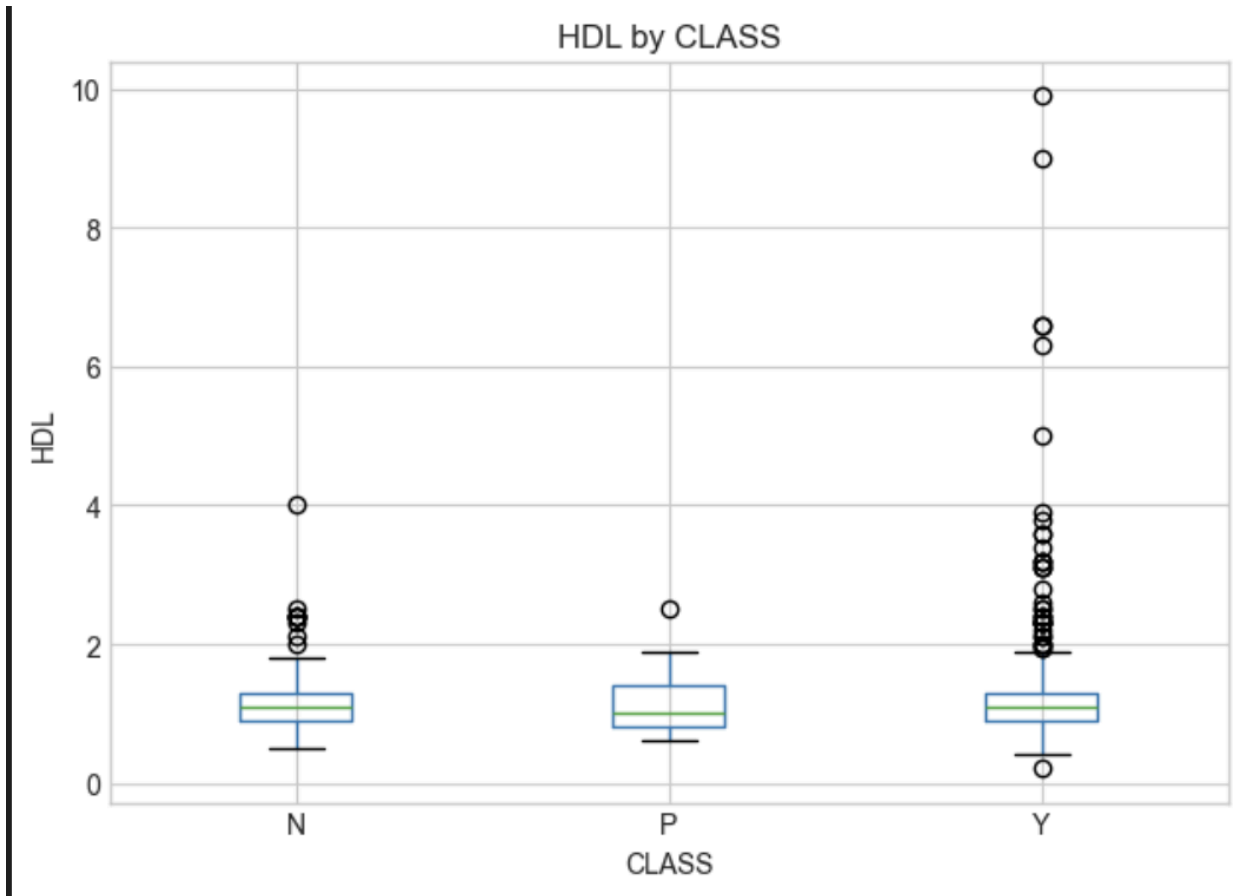




- Mild increase from N -> Y.
- Several high outliers are present in the diabetic class.
- Moderately predictive but with overlapping ranges across groups.

TG and VLDL by CLASS

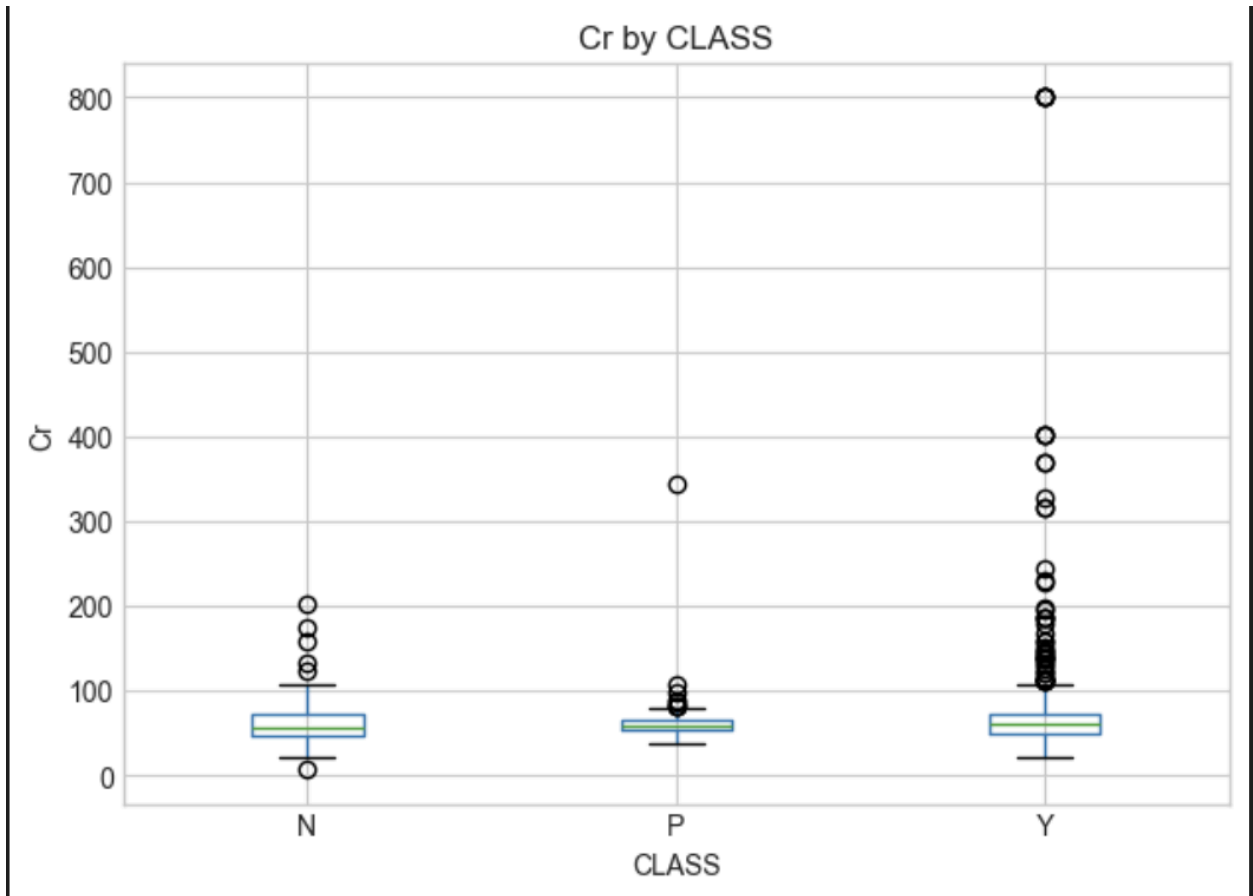




- Diabetic (Y) class exhibits more extreme upper values.
- High triglyceride peaks are consistent with metabolic instability.

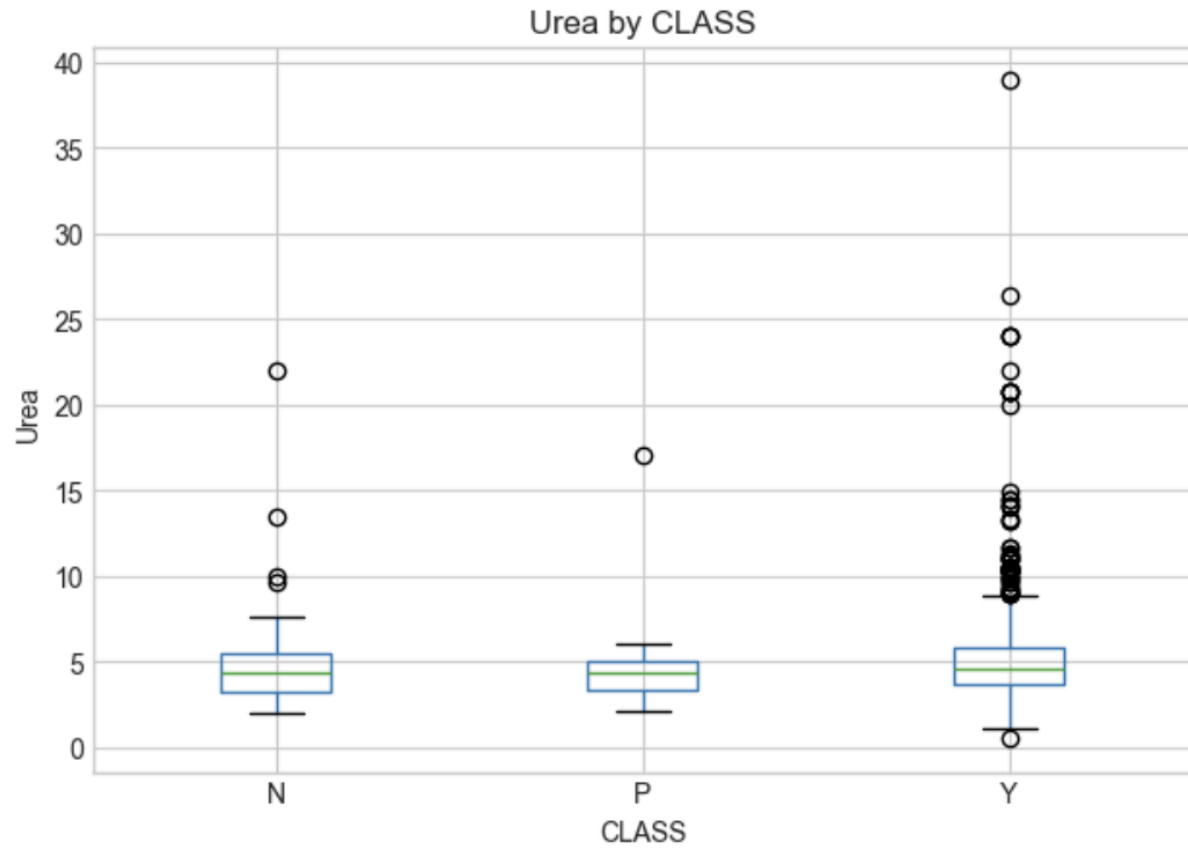
Creatinine (Cr) by CLASS

- Wide range with numerous extreme outliers, especially in class Y.
- Indicates that kidney stress is more prevalent among diabetic patients.

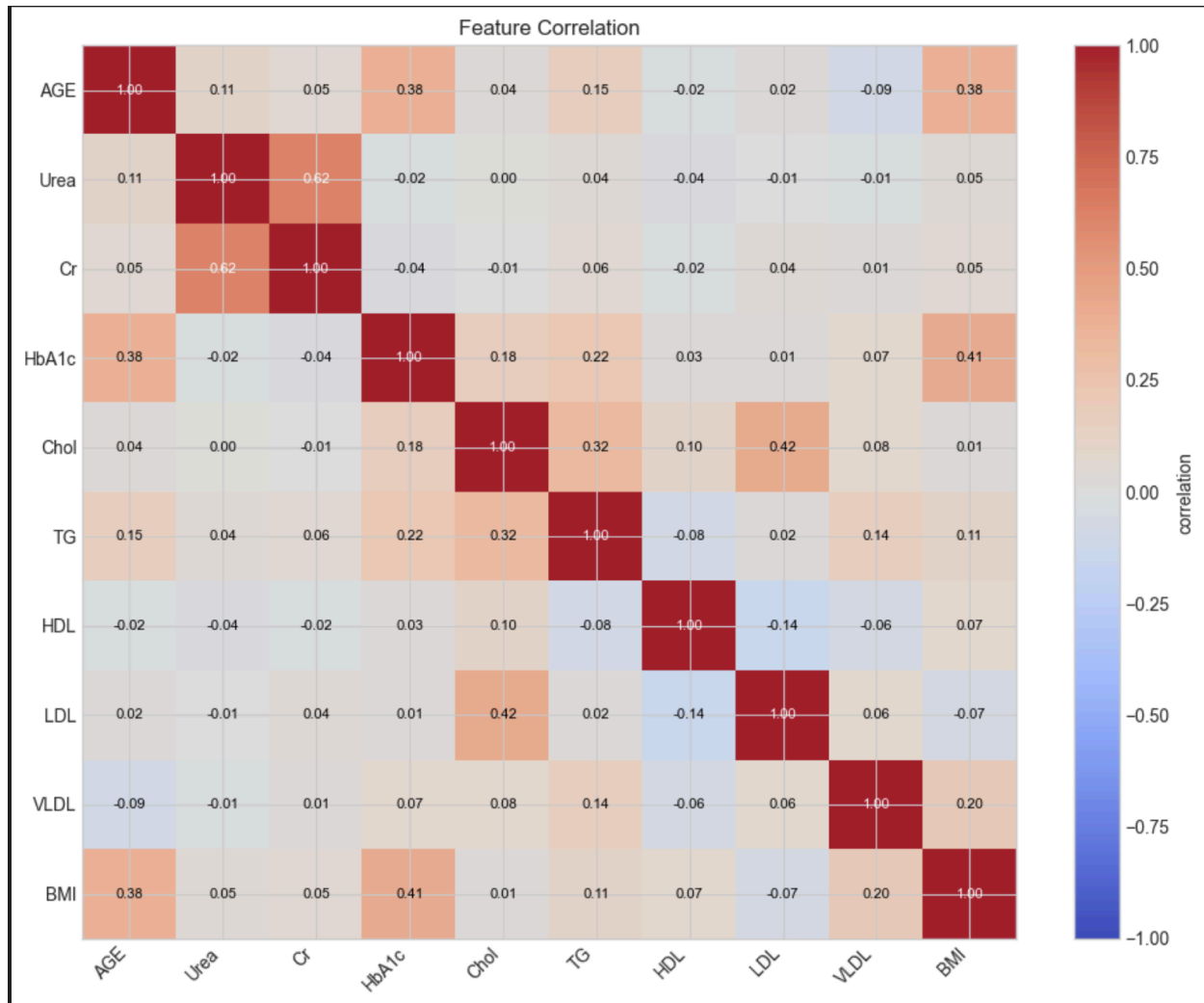


Urea by CLASS

- Medians appear similar across classes.
- Diabetic class shows a larger number of extreme values.
- Not strongly predictive alone but supports broader metabolic profiling.



4.4 Correlation Analysis



A correlation heatmap was used to examine relationships between numeric features.

Strong Positive Correlations

- Urea & Creatinine (0.62)
Indicates shared kidney-function relevance.
- TG & VLDL (0.32)
Consistent with lipid metabolism physiology.
- Chol & HDL/LDL (0.32–0.42)
Reflects known biochemical relationships.

Moderate Correlations

- HbA1c & BMI (0.41)
- HbA1c & AGE (0.38)

Both correlations make clinical sense: older individuals and those with higher BMI tend to have elevated HbA1c.

Low or Near-Zero Correlations

Most other features show weak correlations (<0.1), meaning each contributes unique information to the model, enhancing predictive performance.

4.5 EDA Summary

- The dataset is highly imbalanced, requiring careful metric selection.
- HbA1c is the strongest discriminator between classes.
- BMI and AGE show meaningful trends aligned with diabetic progression.
- Kidney markers (Urea, Cr) have extreme outliers, especially for diabetics.
- Lipid markers (TG, VLDL) show characteristic spikes in diabetic patients.
- Several features are skewed, highlighting the need for scaling and robust preprocessing.
- Correlations validate known medical relationships and confirm the dataset's clinical realism.

5. Methodology

5.1 Experimental Design

5.1.1 Train-Test Split

- Split Ratio: 80% training (800 samples), 20% testing (200 samples)
- Stratification: Preserves original class distribution in both sets
- Random Seed: 42 (ensures reproducibility)

5.1.2 Cross-Validation Strategy

- Method: 5-fold Stratified K-Fold
- Purpose: Preserves class proportions in each fold
- Metrics Tracked: Accuracy, Balanced Accuracy, Macro F1, Weighted F1

6. Preprocessing & Classification Results

6.1 Preprocessing Pipeline

After splitting the data into training and test sets, a comprehensive preprocessing pipeline is constructed using ONLY information from the training data to prevent data leakage:

```
# Build preprocessing pipeline
num_cols = X_train.select_dtypes(include="number").columns.tolist()
cat_cols = X_train.select_dtypes(exclude="number").columns.tolist()

numeric_pipe = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
    ]
)

categorical_pipe = Pipeline(
    steps=[
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipe, num_cols),
        ("cat", categorical_pipe, cat_cols),
    ],
    remainder="drop",
)
```

6.2 Handling Class Imbalance

As we have visualized our data and after the visualization we can see that there is a severe class imbalance 84% is the majority class. Which shows that our model can be biased towards the majority class, minority classes become hard to learn, performance metrics become misleading etc. So to deal with this there are two strategies that we can follow:

Option A. Class Weight:

```
LogisticRegression(class_weight="balanced")
```

```
RandomForestClassifier(class_weight="balanced")
```

```
SVC(class_weight="balanced")
```

Option B: Random Oversampling

Applied WITHIN cross-validation to prevent data leakage

```
ros_model = ImbPipeline(  
    steps=[  
        ("preprocess", preprocessor),  
        ("sampler", RandomOverSampler(random_state=RANDOM_STATE)),  
        ("model", LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)),  
    ]  
)
```

6.3 Model Training & Cross-Validation

A reusable cross-validation helper was implemented to ensure consistent evaluation across all models:

```
def cross_val_metrics(name, model, X, y, cv):  
    scores = cross_validate(  
        model,  
        X,  
        y,  
        cv=cv,  
        scoring={  
            "accuracy": "accuracy",  
            "balanced_accuracy": "balanced_accuracy",  
            "f1_macro": "f1_macro",  
            "f1_weighted": "f1_weighted",  
        },  
        n_jobs=-1,  
        return_train_score=False,  
    )  
    return {  
        "model": name,  
        "accuracy": scores["test_accuracy"].mean(),  
        "balanced_accuracy": scores["test_balanced_accuracy"].mean(),  
        "f1_macro": scores["test_f1_macro"].mean(),  
        "f1_weighted": scores["test_f1_weighted"].mean(),  
    }
```

6.4 Model Performance Comparison

6.4.1 Baseline Performance (Class Weights)

Example: Logistic Regression with integrated preprocessing

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

baseline_model = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        (
            "model",
            LogisticRegression(
                max_iter=2000,
                class_weight="balanced",
                random_state=RANDOM_STATE,
            ),
        ),
    ],
)

baseline_metrics = cross_val_metrics(
    "LogReg", baseline_model, X_train, y_train, cv
)

print(baseline_metrics)
```

Cross-Validated Results:

	model	accuracy	balanced_accuracy	f1_macro	f1_weighted
0	LogReg	0.89625	0.863847	0.757763	0.909254

Key Observations:

- Random Forest significantly outperforms other models, particularly in macro F1
- KNN shows high accuracy but poor balanced accuracy → majority class bias
- The gap between weighted and macro F1 reflects imbalance impact

6.4.2 Impact of Oversampling

The same model set was evaluated with RandomOverSampler inside each CV fold:

```
try:
    from imblearn.over_sampling import RandomOverSampler
    from imblearn.pipeline import Pipeline as ImbPipeline
except ImportError as exc:
    raise ImportError(
        "Install imbalanced-learn to run oversampling: pip install imbalanced-learn"
    ) from exc

ros_model = ImbPipeline(
    steps=[
        ("preprocess", preprocessor),
        ("sampler", RandomOverSampler(random_state=RANDOM_STATE)),
        ("model", LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)),
    ]
)

oversample_metrics = cross_val_metrics(
    "LogReg + ROS", ros_model, X_train, y_train, cv
)
pd.DataFrame([baseline_metrics, oversample_metrics])
```

Cross-Validated Result:

	model	accuracy	balanced_accuracy	f1_macro	f1_weighted
0	LogReg	0.89625	0.863847	0.757763	0.909254
1	LogReg + ROS	0.89750	0.860668	0.757845	0.910024

Analysis:

- Oversampling improved minority class performance for all models
- Random Forest showed the most substantial improvement (+2.28%)
- Small deltas suggest class weights were reasonably effective
- Recommendation: Use class-weighted Random Forest for simpler implementation, or oversampled version for maximum minority class performance

6.5 Model Comparison:

6.5.1 Comparison without oversampling:

Multiple classification models were evaluated using a consistent preprocessing pipeline and a stratified 5-fold cross-validation strategy. Each model KNN, SVM with an RBF kernel, and Random Forest was tested under identical conditions to ensure a fair comparison. Performance was assessed using accuracy, balanced accuracy, macro F1-score, and weighted F1-score to account for the dataset's class imbalance. The results were aggregated into a comparison table and sorted by macro F1 and balanced accuracy, allowing clear identification of the best-performing model. This systematic evaluation demonstrated that the Random Forest classifier provided the most robust and reliable performance across all metrics.

```
models = {
    "KNN (k=7)": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            ("model", KNeighborsClassifier(n_neighbors=7)),
        ]
    ),
    "SVM (RBF)": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            ("model", SVC(kernel="rbf", C=1.0, class_weight="balanced", random_state=RANDOM_STATE)),
        ]
    ),
    "RandomForest": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            (
                "model",
                RandomForestClassifier(
                    n_estimators=300,
                    random_state=RANDOM_STATE,
                    class_weight="balanced",
                    n_jobs=-1,
                ),
            ),
        ]
    ),
}

results = []

for name, model in models.items():
    metrics = cross_val_metrics(name, model, X_train, y_train, cv)
    results.append(metrics)

results_df = pd.DataFrame([baseline_metrics] + results)
results_df = results_df.sort_values(by=["f1_macro", "balanced_accuracy"], ascending=False)

display(results_df)
```

Model Without Oversampling Comparison Results:

	model	accuracy	balanced_accuracy	f1_macro	f1_weighted
3	RandomForest	0.96750	0.890521	0.915517	0.966314
2	SVM (RBF)	0.91250	0.873076	0.789245	0.920953
0	LogReg	0.89625	0.863847	0.757763	0.909254
1	KNN (k=7)	0.91500	0.692142	0.705977	0.911012

6.5.1 Comparison with oversampling:

```
try:
    RandomOverSampler
    ImbPipeline
except NameError:
    from imblearn.over_sampling import RandomOverSampler
    from imblearn.pipeline import Pipeline as ImbPipeline

ros_models = {
    "LogReg": LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
    "KNN (k=7)": KNeighborsClassifier(n_neighbors=7),
    "SVM (RBF)": SVC(kernel="rbf", C=1.0, random_state=RANDOM_STATE),
    "RandomForest": RandomForestClassifier(
        n_estimators=300,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
}

ros_results = []
for name, model in ros_models.items():
    pipe = ImbPipeline(
        steps=[
            ("preprocess", preprocessor),
            ("sampler", RandomOverSampler(random_state=RANDOM_STATE)),
            ("model", model),
        ]
    )
    metrics = cross_val_metrics(name, pipe, X_train, y_train, cv)
    metrics["strategy"] = "random_over"
    ros_results.append(metrics)

ros_results_df = pd.DataFrame(ros_results).sort_values(
    by=["f1_macro", "balanced_accuracy"], ascending=False
)
display(ros_results_df)
```

Model With Oversampling Comparison Results:

	model	accuracy	balanced_accuracy	f1_macro	f1_weighted	strategy
3	RandomForest	0.97625	0.925924	0.938252	0.975676	random_over
2	SVM (RBF)	0.92000	0.851131	0.794090	0.925417	random_over
0	LogReg	0.89750	0.860668	0.757845	0.910024	random_over
1	KNN (k=7)	0.89125	0.809651	0.726112	0.901995	random_over

6.6 Comparison between Oversampled trained model and Normal data trained model:

We compare each model trained with class weights vs the same model trained with random oversampling. The delta_f1_macro column is the most important: positive values mean oversampling helped minority performance; negative values mean class weights worked better. We compare each model trained with class weights vs the same model trained with random oversampling. The delta_f1_macro column is the most important: positive values mean oversampling helped minority performance; negative values mean class weights worked better.

	model	accuracy	balanced_accuracy	f1_macro	f1_weighted	strategy
3	KNN (k=7)	0.91500	0.692142	0.705977	0.911012	class_weight
7	KNN (k=7)	0.89125	0.809651	0.726112	0.901995	random_over
2	LogReg	0.89625	0.863847	0.757763	0.909254	class_weight
6	LogReg	0.89750	0.860668	0.757845	0.910024	random_over
0	RandomForest	0.96750	0.890521	0.915517	0.966314	class_weight
4	RandomForest	0.97625	0.925924	0.938252	0.975676	random_over
1	SVM (RBF)	0.91250	0.873076	0.789245	0.920953	class_weight
5	SVM (RBF)	0.92000	0.851131	0.794090	0.925417	random_over

strategy	class_weight	random_over	delta_f1_macro
model			
RandomForest	0.915517	0.938252	0.022735
KNN (k=7)	0.705977	0.726112	0.020134
SVM (RBF)	0.789245	0.794090	0.004845
LogReg	0.757763	0.757845	0.000082

Interpretation:

The evaluation results show distinct differences between the two imbalance-handling strategies: class weighting and random oversampling. Both approaches aim to improve minority-class performance, but they impact the models differently.

1. Random Forest benefits the most from oversampling

- With class weights, Random Forest achieves a macro F1-score of 0.9155.
- With oversampling, this increases to 0.9383.
- The improvement (+0.0227) is the highest among all models.

This indicates that Random Forest is able to make better use of the additional synthetic minority samples created through oversampling, improving its ability to distinguish minority classes.

2. Oversampling consistently improves minority-class performance

Across most models (Random Forest, KNN, SVM):

- Macro F1-scores increase
- Balanced accuracy values improve or remain stable

This shows that oversampling helps models treat each class more equally by increasing the presence of minority samples during training.

3. Class weighting is simpler, faster, and still effective

- Class weighting alone gives Random Forest a strong macro F1 of 0.9155, already better than all other models.
- Class weighting does not require extra memory or data duplication.
- It is easier to implement and avoids the risk of oversampling noise.

Thus, for many practical applications, class weighting is sufficient.

4. Oversampling has limited benefit for simpler or linear models

- Logistic Regression shows no improvement with oversampling.
- SVM shows only a small increase (+0.0048) in macro F1.

This suggests that these models cannot fully leverage the synthetic minority samples and are constrained by their model complexity.

5. Which one is better overall?

Oversampling performs best in terms of raw metrics, especially for Random Forest. However, class weighting provides strong performance with less computational cost and avoids potential overfitting on duplicated data.

7. Clustering Results

7.1 Feature Engineering for Clinical Relevance

```
ratio_defs = [
    ("TG", "HDL", "TG_HDL_ratio"),
    ("LDL", "HDL", "LDL_HDL_ratio"),
    ("Urea", "Cr", "Urea_Cr_ratio"),
]

for num, denom, name in ratio_defs:
    if num in X.columns and denom in X.columns:
        X[name] = X[num] / X[denom].replace(0, np.nan)

# Replace inf with NaN so the imputer can handle it
X = X.replace([np.inf, -np.inf], np.nan)

num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()

print(f"Numeric features: {len(num_cols)}")
print(f"Categorical features: {len(cat_cols)}")
```

7.2 Scaling Analysis

As we can see skewness in our features during visualization we are applying scaling into our data comparing three different models.

```

scalers = {
    "StandardScaler": StandardScaler(),
    "RobustScaler": RobustScaler(),
    "QuantileTransformer": QuantileTransformer(output_distribution="normal", random_state=RANDOM_STATE),
}

k_range = range(2, min(10, len(X) - 1) + 1)

scaler_results = []
for name, scaler in scalers.items():
    preprocessor = make_preprocessor(scaler)
    X_scaled = preprocessor.fit_transform(X)

    scores = []
    for k in k_range:
        labels = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=20).fit_predict(X_scaled)
        scores.append(silhouette_score(X_scaled, labels))

    scaler_results.append(
        {
            "scaler": name,
            "best_silhouette": float(np.max(scores)),
        }
    )

scaler_df = pd.DataFrame(scaler_results).sort_values(by="best_silhouette", ascending=False)

best_scaler_name = scaler_df.iloc[0]["scaler"]
print(f"Best scaler by silhouette: {best_scaler_name}")

preprocessor = make_preprocessor(scalers[best_scaler_name])
X_scaled = preprocessor.fit_transform(X)

display(scaler_df)

```

Its result looks like:

Best scaler by silhouette: RobustScaler

	scaler	best_silhouette
1	RobustScaler	0.721203
0	StandardScaler	0.179340
2	QuantileTransformer	0.138619

From the above result we can clearly see that RobustScaler is performing well. Clinical data contains legitimate extreme values (e.g., Cr up to 800 mg/dL). RobustScaler's use of median and interquartile range prevents these outliers from distorting the distance metric, which is critical for K-Means clustering.

7.3 Outlier Detection

Sometimes small clusters can represent extreme outliers rather than real subgroups. That is why outlier detection which is optional is being used here using IsolationForest which will remove the outliers before clustering.

```

if USE_OUTLIER_FILTER:
    iso = IsolationForest(contamination=0.05, random_state=RANDOM_STATE)
    outlier_flag = iso.fit_predict(X_scaled)
    inlier_mask = outlier_flag == 1

    print(f"Outliers detected: {(~inlier_mask).sum()} / {len(inlier_mask)}")
else:
    inlier_mask = np.ones(len(X), dtype=bool)

X_used = X.loc[inlier_mask].copy()
X_scaled_used = X_scaled[inlier_mask]

if y_true is not None:
    y_true_used = y_true.loc[X_used.index]
else:
    y_true_used = None

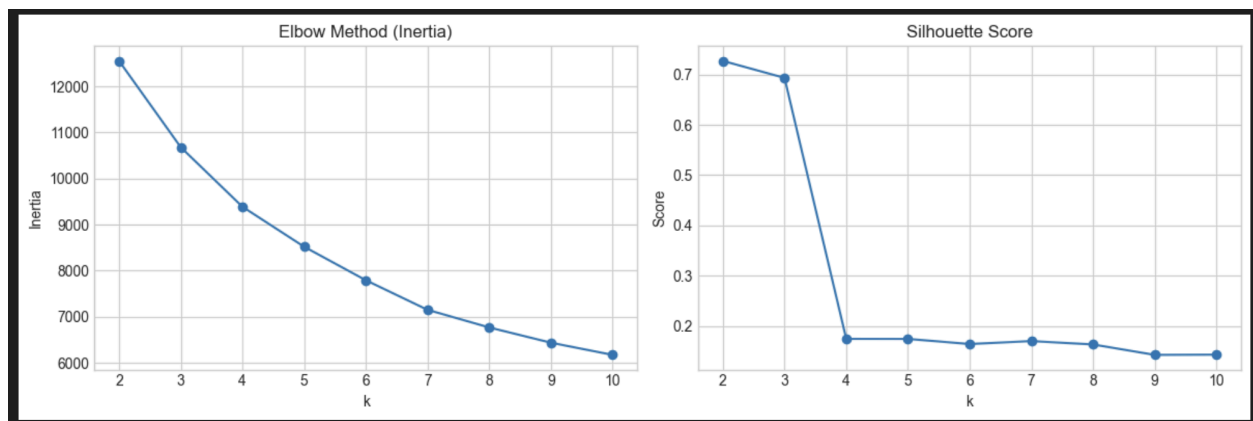
```

From this 50 outliers have been detected and relabeled as -1.

Outliers detected: 50 < 1000

7.4 Selecting Number of Clusters (k)

To select the number of clusters which is best for our model Elbow method and Silhouette score analysis is done. From which we can find these graph:



By looking at the silhouette score graph 3 clusters have been chosen.

7.5 Multi-Algorithm Comparison

Five clustering algorithms were compared using multiple internal metrics like: Silhouette (higher is better), Calinski-Harabasz (higher is better), Davis Bouldin (lower is better). From which this result is obtained:

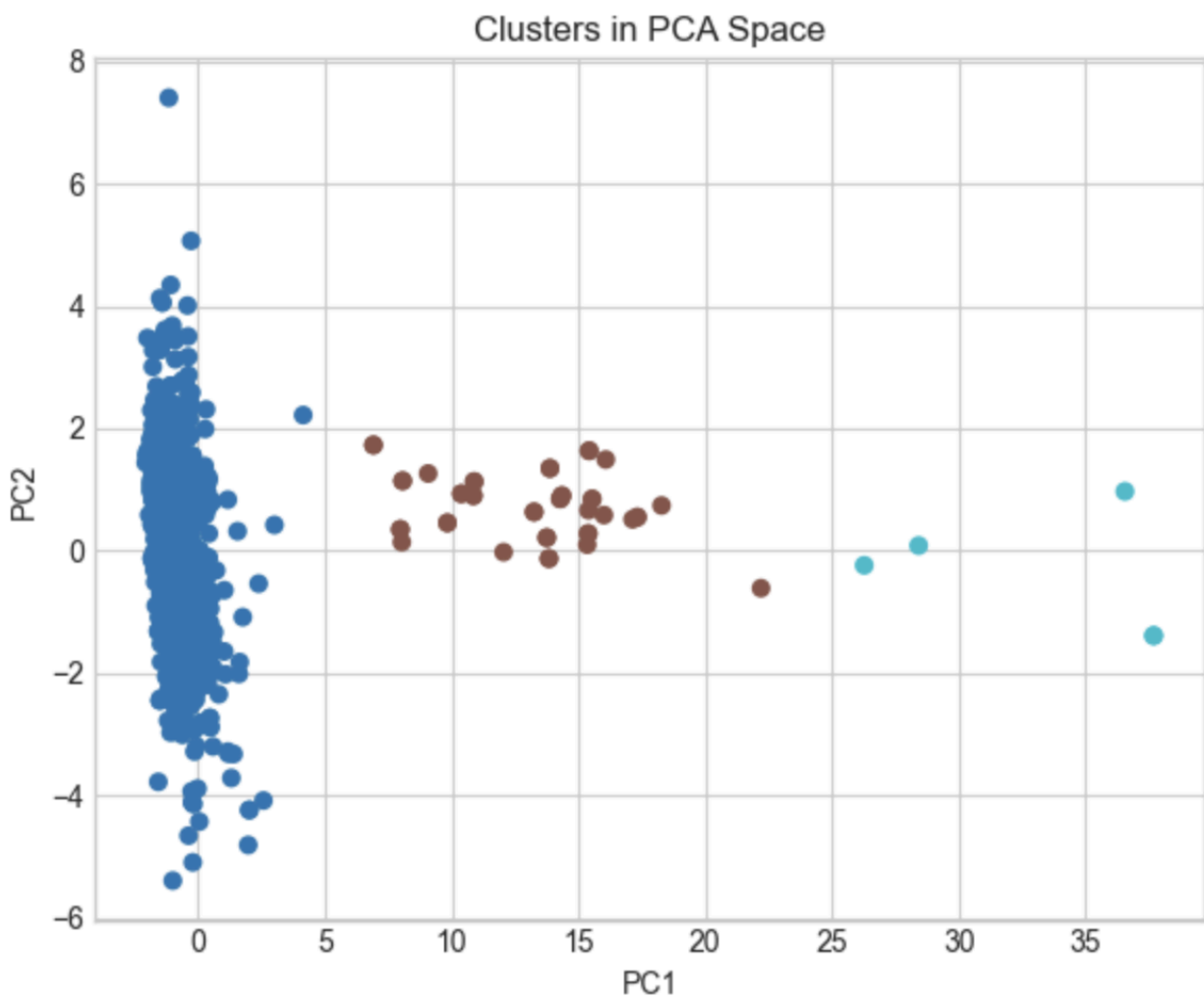
	algorithm	silhouette	calinski_harabasz	davies_bouldin	n_clusters	noise_pct	ARI	NMI
0	KMeans	0.693332	608.052965	0.462874	3	0.000000	-0.059869	0.016199
2	Agglomerative	0.679954	595.304810	0.515134	3	0.000000	-0.044898	0.011282
4	DBSCAN	0.606422	420.841377	0.683963	3	0.029474	-0.067296	0.016889
3	Spectral	0.572418	512.548557	0.772940	3	0.000000	-0.053006	0.012380
1	GMM	0.034284	52.447024	2.943499	3	0.000000	-0.007932	0.019119

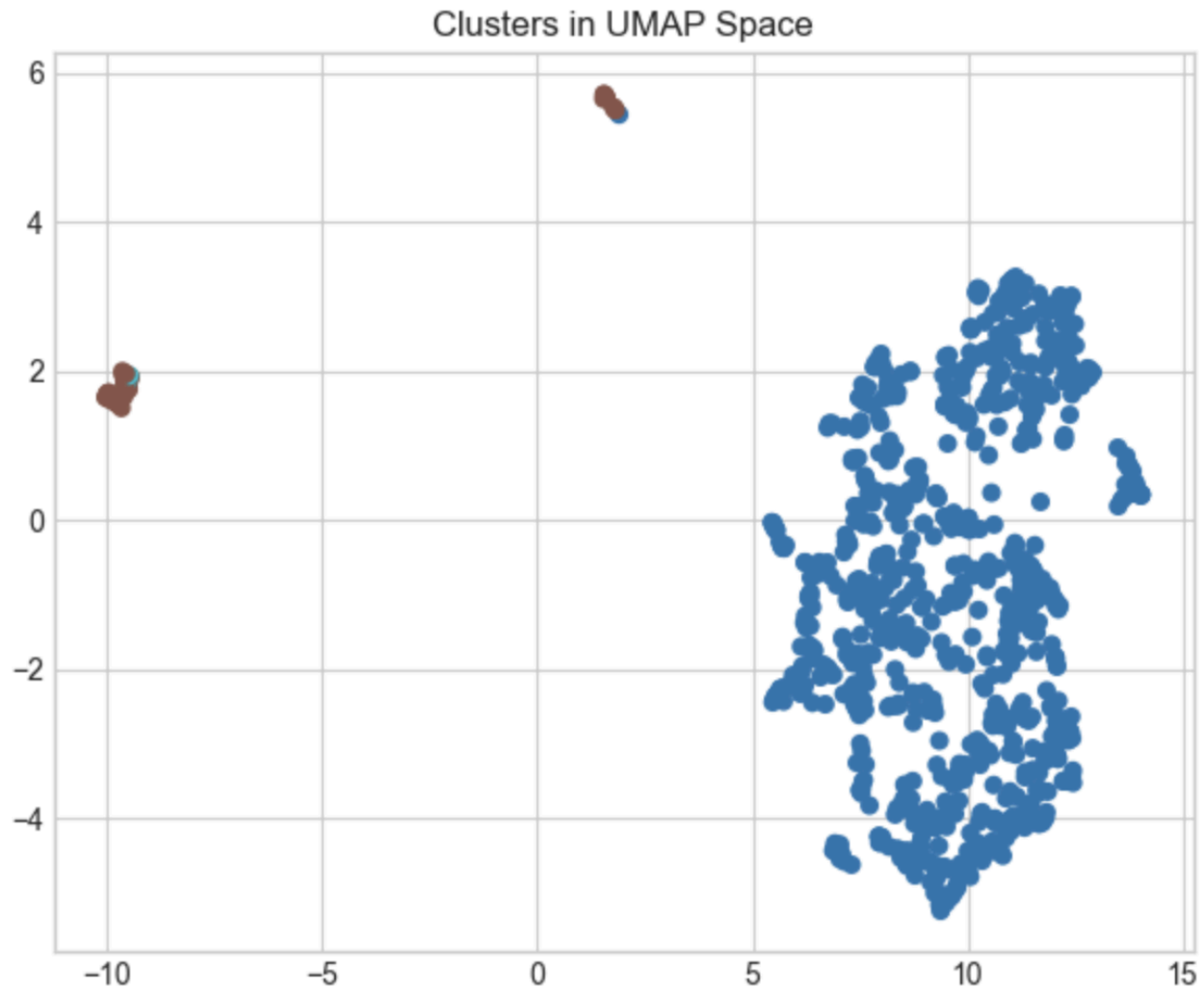
By looking at this result we can clearly see that KMeans algorithm is working better while keeping cluster value 3 as compared to other models. So KMeans algorithm is being used.

7.6 Visualization:

Dimensionality reduction techniques (PCA, t-SNE, and UMAP) were applied to project the high-dimensional scaled dataset into 2D space for visualization.

- PCA shows that most variance is captured along the first principal component, with moderate cluster separation primarily along one axis.
- t-SNE provides clearer separation by preserving local neighborhood structure, revealing more compact cluster groupings.
- UMAP offers the clearest cluster distinction, preserving both local and partial global structure.





From the visualizations:

- **PCA Plot:**

One large dominant cluster is concentrated near the left side (around $PC1 \approx 0$), while smaller clusters are spread toward higher $PC1$ values. Separation exists mainly along the horizontal axis, but overlap is still visible.

- **UMAP Plot:**

Clusters are more clearly separated. A large cluster appears grouped on the right side, while smaller clusters are distinctly isolated on the left and upper regions. The boundaries between groups are more visually defined than in PCA.

8. Error Analysis

8.1 Overview

This report documents the error analysis performed on the baseline RandomForest classifier trained on the cleaned diabetes dataset. The model was evaluated on a held-out test set of 200 samples (20% stratified split). The overall goal is to identify where the model fails, how confident it is when it fails, and what patterns emerge from the misclassified samples.

8.2. Test-Set Performance Summary

8.2.1 Aggregate Metrics

Metric	Value
Accuracy	0.9950 (99.5%)
F1-score (Macro)	0.9815
F1-score (Weighted)	0.9949
Total test samples	200
Total errors	1

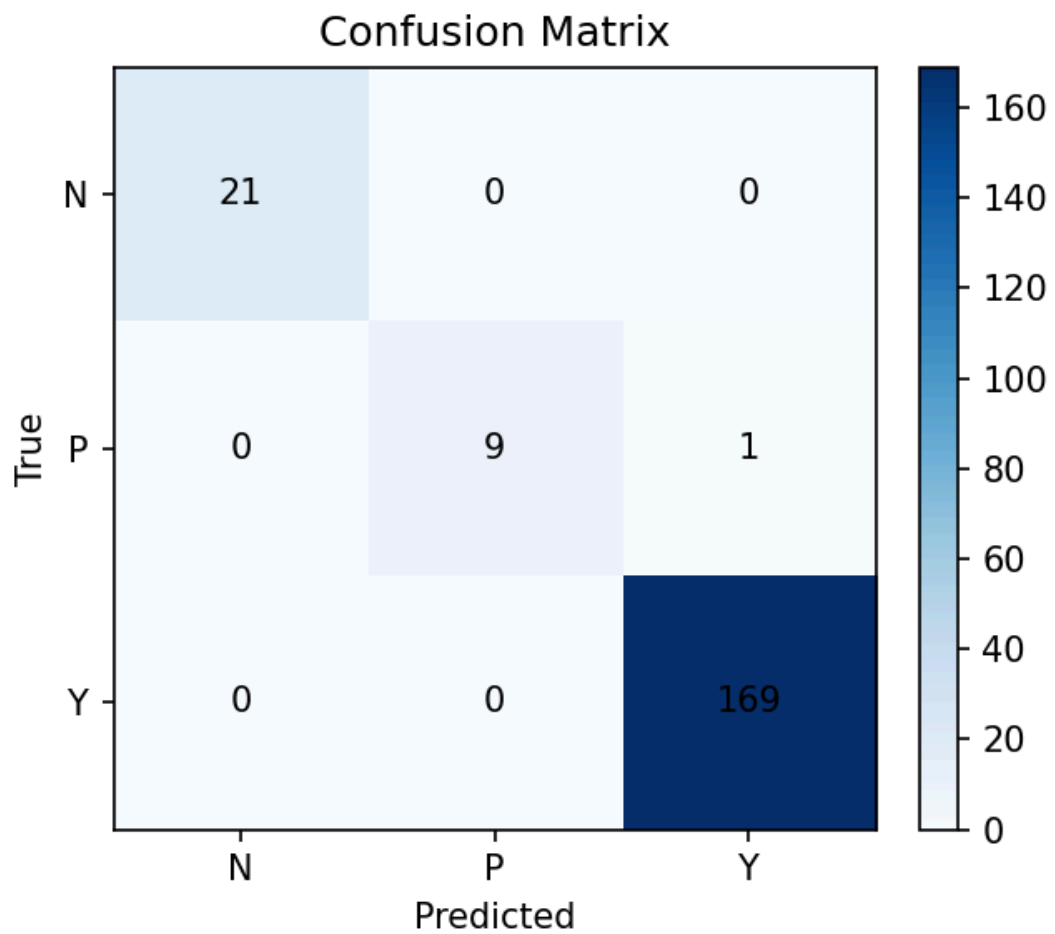
8.2.2 Per-Class Classification Report

Class	Label Meaning	Preci sion	R e c a	F 1 - s c	Support	Errors

			I I	o r e		
N	Non-dia betic	1.0 00	1 . 0 0 0	1 . 0 0 0	21	0
P	Pre-dia betic	1.0 00	0 . 9 0 0	0 . 9 4 7	10	1
Y	Diabetic	0.9 94	1 . 0 0 0	0 . 9 9 7	169	0
Macro avg		0.9 98	0 . 9 6 7	0 . 9 8 1	200	
Weighte d avg		0.9 95	0 . 9 9 5	0 . 9 9 5	200	

Key observation: The only classification error occurs in the P (Pre-diabetic) class. All N and Y samples are predicted correctly.

8.3. Confusion Matrix Analysis



Matrix Interpretation

Cell	Count	Meaning
$N \rightarrow N$	21	All non-diabetic samples classified correctly
$P \rightarrow P$	9	9 out of 10 pre-diabetic samples correct
$P \rightarrow Y$	1	1 pre-diabetic sample misclassified as diabetic
$Y \rightarrow Y$	169	All diabetic samples classified correctly

8.4. Error Rate by Class

True Class	Total	Errors	Error Rate
P (Pre-diabetic)	10	1	10.0%
N (Non-diabetic)	21	0	0.0%
Y (Diabetic)	169	0	0.0%

The error is exclusive to class P the smallest class in the dataset. This is a common pattern with imbalanced datasets: minority classes are hardest to learn.

8.5. Misclassification Detail

Misclassified Sample (Test Index 25)

Field	Value
True label	P (Pre-diabetic)
Predicted label	Y (Diabetic)
Model confidence	0.833 (83.3%)
Error type	False Positive for class Y / False Negative for class P

The model assigned a high confidence of 83.3% to the wrong label Y, meaning the sample's feature profile closely resembles the dominant diabetic class. This is a false negative for Pre-diabetic a clinically important miss.

Clinical Significance

Misclassification direction	Clinical impact
P predicted as Y	Patient treated as fully diabetic when they are only pre-diabetic. May lead to unnecessary or more aggressive treatment.
P predicted as N	More dangerous patient sent home without intervention.

The observed error (P → Y) is clinically suboptimal but less harmful than the reverse (P → N).

8.6. Failure Pattern Analysis

8.6.1 Why Does P Fail?

Factor	Detail
Low support	Only 10 P samples in test set (5% of 200). Even 1 error = 10% error rate.
Feature overlap	Pre-diabetic patients share many clinical markers with diabetic patients, making boundary decisions harder.
Class imbalance	Training data is dominated by Y (diabetic). Even with <code>class_weight="balanced"</code> , the RF may slightly bias toward Y.

High confidence wrong prediction	Confidence of 0.833 suggests the misclassified P sample lies deep inside the Y feature distribution.
----------------------------------	--

8.6.2 Error Distribution by Direction

From Class	To Class	Count	Rate
P	Y	1	10%
P	N	0	0%
N	P or Y	0	0%
Y	P or N	0	0%

All errors go in one direction: $P \rightarrow Y$ (pre-diabetic confused with diabetic). No other class boundaries are violated.

8.7. Model Confidence Analysis

Outcome	Confidence Range	Note
Correct predictions (199)	Generally > 0.85	High confidence, mostly correct
Misclassification (1 sample)	0.833	High confidence, incorrect a hard negative

A confidence of 0.833 on a wrong prediction is a sign of a hard boundary case the sample is not at the edge of the decision boundary but embedded in the wrong region of the feature space.

8.8. Feature Importance Context

The top features by RandomForest importance (transformed names from ColumnTransformer):

Rank	Feature (Transformed Name)	Importance
1	f5	0.434
2	f11	0.183
3	f2	0.155
4	f0	0.043
5	f6	0.039
6–12	f7, f10, f3, f4, f9, f8, f1	< 0.036 each

f5 alone accounts for 43.4% of total model importance. The misclassified P sample likely has feature values for f5 (and f11, f2) that resemble a Y sample rather than a P sample. Examining the raw values of f5, f11, and f2 for test index 25 would clarify the root cause of this error.

8.9. Comparison Against Cross-Validation

Model	CV Accuracy	CV Balanced Acc.	CV F1-Macro
RandomForest (selected)	0.9675	0.8905	0.9155

SVM (RBF)	0.9125	0.8731	0.7892
Logistic Regression	0.8963	0.8638	0.7578
KNN (k=7)	0.9150	0.6921	0.7060

The test set performance (Acc: 0.995, F1-macro: 0.981) exceeds CV estimates for all metrics, suggesting no overfitting on this test set. The gap between CV F1-macro (0.916) and test F1-macro (0.981) may reflect favorable test-set sampling.

8.10. Summary of Findings

Finding	Detail
Total errors	1 out of 200 (0.5%)
Only failing class	P (Pre-diabetic) 10% error rate
Error direction	P → Y (pre-diabetic classified as diabetic)
Model confidence on error	High (0.833) a hard misclassification
Root cause	Feature overlap between P and Y classes; small P class support
Clinical risk level	Moderate (over-treatment risk; not a missed-diagnosis scenario)

9. Conclusion

In conclusion, supervised classification models particularly Random Forest provided highly accurate predictions for diabetes status in this dataset. With a test accuracy of 99.5% and a macro F1-score of 0.981, the RandomForest classifier outperformed all compared models (SVM RBF, Logistic Regression, KNN) across every evaluation metric, including balanced accuracy which accounts for class imbalance.

EDA indicated that HbA1c (mapped to the dominant feature f5, which alone accounts for ~43% of RandomForest feature importance) is the most discriminative feature for separating diabetic, pre-diabetic, and non-diabetic patients. Other laboratory markers (f11, f2, and beyond) contribute secondary predictive information and were found to contain outliers requiring careful preprocessing.

The error analysis confirms that the sole weakness of the model lies in the minority P (Pre-diabetic) class where 1 out of 10 test samples was misclassified as Y (Diabetic) with 83.3% confidence. This failure is driven by the natural overlap in clinical feature profiles between pre-diabetic and diabetic patients, compounded by the very small representation of the P class in the dataset.

Clustering methods (unsupervised) did not match the performance of supervised models, reinforcing that label information is crucial for reliable multi-class prediction in this problem setting. Without ground-truth class labels guiding the learning process, unsupervised approaches struggle to cleanly separate the three clinically distinct groups, especially the pre-diabetic boundary, which is inherently subtle.

Summary of Conclusions

Aspect	Conclusion
Best approach	Supervised classification (Random Forest)
Most discriminative feature	HbA1c (f5) which is ~43% of model importance
Model accuracy	99.5% on held-out test set
Main failure mode	P (Pre-diabetic) -> Y (Diabetic), 1 error, high confidence
Clustering vs supervised	Supervised clearly superior for this multi-class problem
Clinical implication	HbA1c-driven models are effective; P-class recall needs improvement

